

The C8051F040DK development kit – general view

Switching the state of a LED

1 Paper’s purpose

By the end of this first laboratory every student should have basic knowledge regarding the architecture of the CIP-51 core and the C8051F040 microcontroller and a general idea regarding the software and hardware integrated development environment.

2 Introduction

The C8051F040 development kit contains:

- 2.1. Development hardware: the main board and the target board,
- 2.2. Development software: IDE (Integrated Development Environment).

2.1 Development hardware

2.1.1 Main board

The main board provides power connectors for the power module, the target board and other dedicated boards. These dedicated boards are powered through the 9V power bus (left) or through the 5V power bus (right). The target board is powered using a different power cable.

2.1.2 The power module

The power module is connected to the power mainframe through a 9V DC adapter. This module purpose is to distribute power to all the other boards as follows:

- 9V to the target board (using a power cable),
- 9V to the first power bus (left) and thus to all dedicated boards connected to the bus,
- 5V to the second power bus (right) and thus to all dedicated boards connected to the bus.

2.1.2.1 Switches and LEDs

- SW1 switch powers the 9V power bus
- SW2 switch powers the 5V power bus
- Led1 signals the power state of the 9V power bus (lit – powered)
- Led2 signals the power state of the 5V power bus (lit – powered)

2.1.2.2 Power connectors

- J3 – input from the power mainframe
- J4 – 9V output to the target board
- 9V connector – powers the 9V power bus
- 5V connector – powers the 5V power bus
- GND – ground connectors for the two power busses

2.1.3.6 System clock sources

The C8051F040 device installed on the target board features a calibrated programmable internal oscillator which is enabled as the system clock source on reset. After reset, the internal oscillator operates at a frequency of 3.0625MHz ($\pm 2\%$) by default but may be configured by software to operate at other frequencies. Therefore, in many applications an external oscillator is not required. However, an external 22.1184MHz crystal is installed on the target board for additional applications. Refer to the C8051F04x data sheet (C8051F04xRev1_4.pdf) for more information on configuring the system clock source.

2.1.3.7 PORT I/O connectors (J12-J19)

In addition to all port I/O signals being routed to the 96-pin expansion connector, each of the eight parallel ports of the C8051F040 has its own 10-pin header connector. Each connector provides a pin for the corresponding port pins 0-7, +3.3VDC and digital ground.

2.1.3.8 Expansion I/O connector (J24)

The 96-pin expansion I/O connector J24 is used to connect daughter boards to the main target board. J24 provides access to many C8051F040 signal pins.

2.1.3.9 VREF connector (J22)

The VREF connector (J22) can be used to connect the VREF (Voltage Reference) output of the C8051F040 to any (or all) of its voltage reference inputs.

2.1.4 System-on-a-chip (SOC)

The C8051F04x family of devices is fully integrated mixed-signal System-on-a-Chip MCUs with 64 digital I/O pins (C8051F040/2/4/6). The structure of the system is depicted in Fig. 2.1.4a and Fig. 2.1.4b.

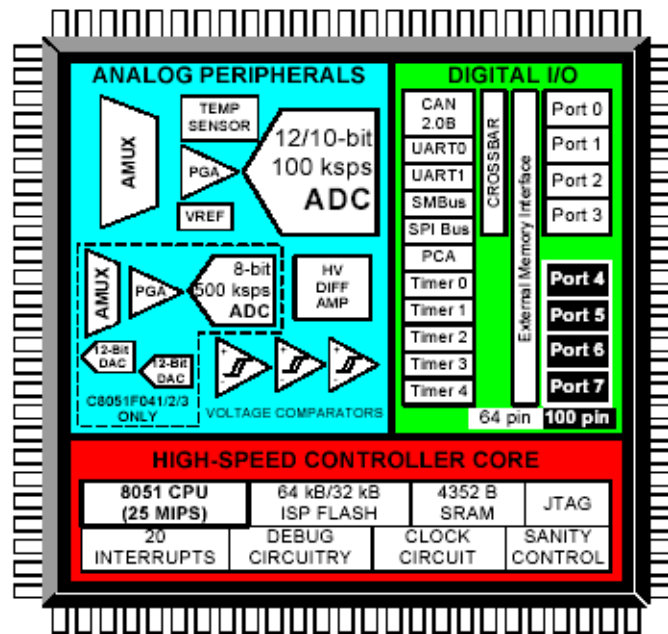


Fig. 2.1.4a SOC Structure

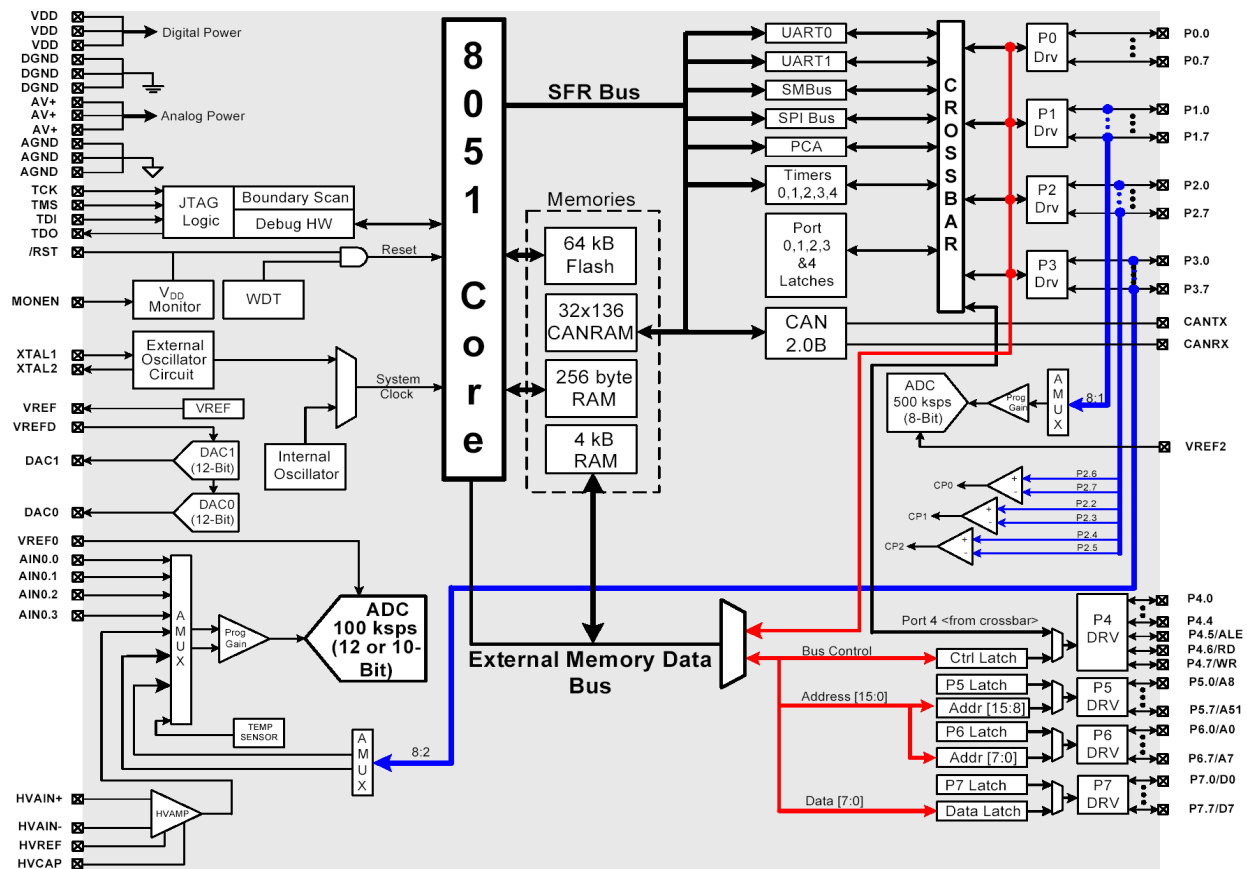


Fig. 2.1.4b C8051F040 Block Diagram

2.1.4.1 CIP-51 Microcontroller Core

2.1.4.1.1 Fully 8051 compatible

The C8051F04x family of devices utilizes Silicon Labs' proprietary CIP-51 microcontroller core. The CIP-51 is fully compatible with the MCS-51™ instruction set. Standard 803x/805x assemblers and compilers can be used to develop software.

2.1.4.1.2 Improved Throughput

The CIP-51 employs a pipelined architecture that greatly increases its instruction throughput over the standard 8051 architecture. In a standard 8051, all instructions except for MUL and DIV take 12 or 24 system clock cycles to execute with a maximum system clock of 12-to-24 MHz. By contrast, the CIP-51 core executes 70% of its instructions in one or two system clock cycles, with only four instructions taking more than four system clock cycles.

With the CIP-51's maximum system clock at 25 MHz, it has a peak throughput of 25 MIPS. Fig. 2.1.4.1.2 shows a comparison of peak throughputs of various 8-bit microcontroller cores with their maximum system clocks.

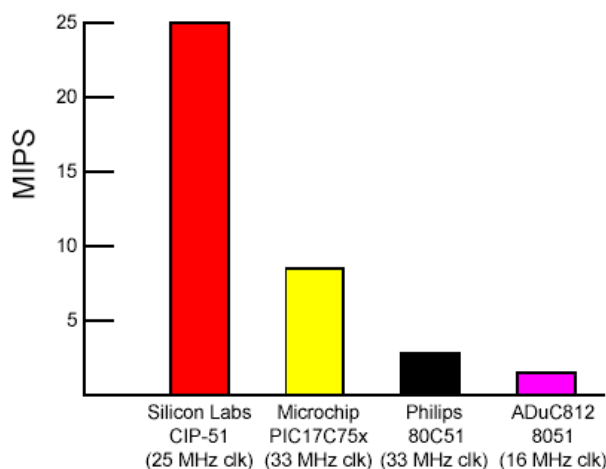


Fig. 2.1.4.1.2 Comparison of Peak CPU Execution Speeds

2.1.4.1.3 Additional features

The C8051F04x MCU family includes several key enhancements to the CIP-51 core and peripherals to improve overall performance and ease of use in end applications. The extended interrupt handler provides 20 interrupt sources into the CIP-51 (as opposed to 7 for the standard 8051), allowing the numerous analog and digital peripherals to interrupt the controller. An interrupt driven system requires less intervention by the MCU, giving it more effective throughput. The extra interrupt sources are very useful when building multi-tasking, real-time systems.

There are up to seven reset sources for the MCU: an on-board VDD monitor, a Watchdog Timer, a missing clock detector, a voltage level detection from Comparator0, a forced software reset, the CNVSTR0 input pin, and the /RST pin.

The MCU has an internal, stand alone clock generator which is used by default as the system clock after any reset. If desired, the clock source may be switched on the fly to the external oscillator, which can use a crystal, ceramic resonator, capacitor, RC, or external clock source to generate the system clock. This can be extremely useful in low power applications, allowing the MCU to run from a slow (power saving) external crystal source, while periodically switching to the fast (up to 25 MHz) internal oscillator as needed.

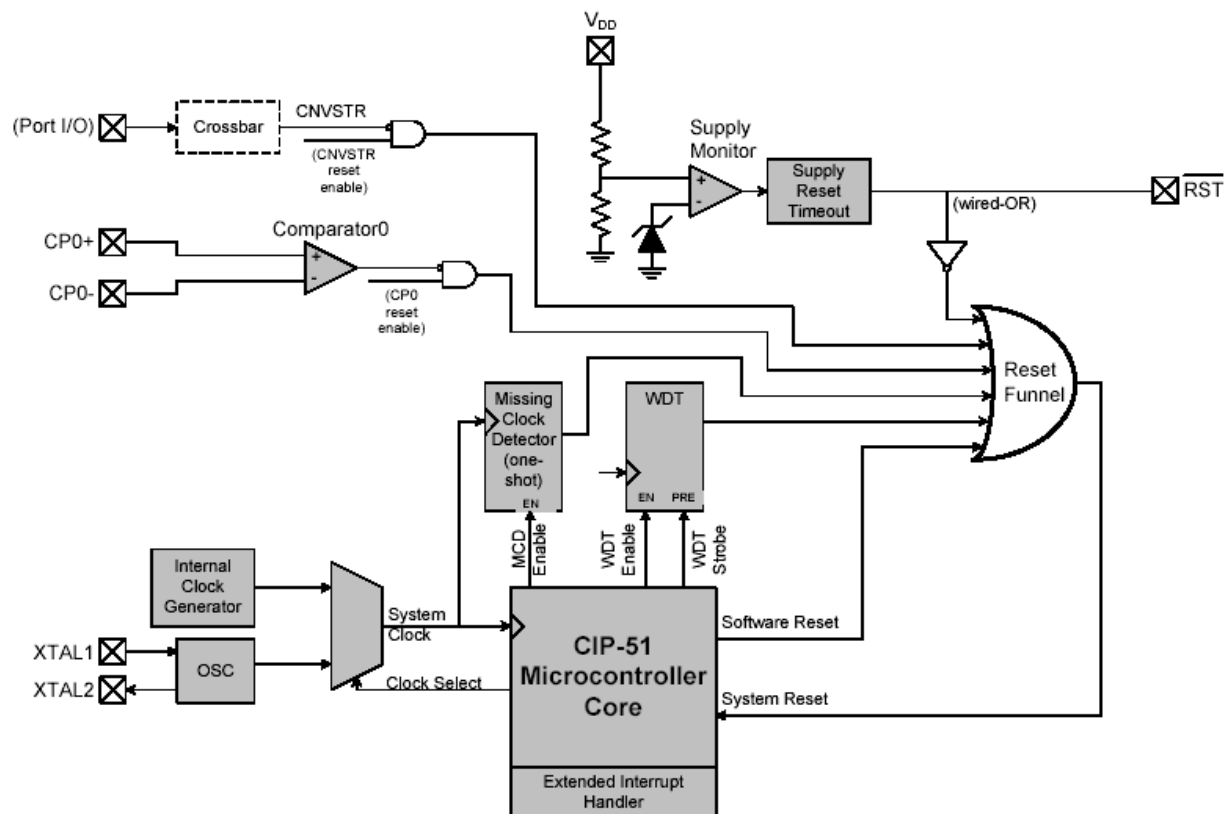


Fig. 2.1.4.1.3 On-Board Clock and Reset

2.1.4.1.4 Instruction set

Table 2.1.4.1.4 The instruction set for the CIP-51 Microcontroller Core.

Mnemonic	Description	Bytes	Clock Cycles
Arithmetic Operations			
ADD A, Rn	Add register to A	1	1
ADD A, direct	Add direct byte to A	2	2
ADD A, @Ri	Add indirect RAM to A	1	2
ADD A, #data	Add immediate to A	2	2
ADDC A, Rn	Add register to A with carry	1	1
ADDC A, direct	Add direct byte to A with carry	2	2
ADDC A, @Ri	Add indirect RAM to A with carry	1	2
ADDC A, #data	Add immediate to A with carry	2	2
SUBB A, Rn	Subtract register from A with borrow	1	1
SUBB A, direct	Subtract direct byte from A with borrow	2	2
SUBB A, @Ri	Subtract indirect RAM from A with borrow	1	2
SUBB A, #data	Subtract immediate from A with borrow	2	2
INC A	Increment A	1	1
INC Rn	Increment register	1	1
INC direct	Increment direct byte	2	2
INC @Ri	Increment indirect RAM	1	2
DEC A	Decrement A	1	1

Mnemonic	Description	Bytes	Clock Cycles
DEC Rn	Decrement register	1	1
DEC direct	Decrement direct byte	2	2
DEC @Ri	Decrement indirect RAM	1	2
INC DPTR	Increment Data Pointer	1	1
MUL AB	Multiply A and B	1	4
DIV AB	Divide A by B	1	8
DA A	Decimal adjust A	1	1
Logical Operations			
ANL A, Rn	AND Register to A	1	1
ANL A, direct	AND direct byte to A	2	2
ANL A, @Ri	AND indirect RAM to A	1	2
ANL A, #data	AND immediate to A	2	2
ANL direct, A	AND A to direct byte	2	2
ANL direct, #data	AND immediate to direct byte	3	3
ORL A, Rn	OR Register to A	1	1
ORL A, direct	OR direct byte to A	2	2
ORL A, @Ri	OR indirect RAM to A	1	2
ORL A, #data	OR immediate to A	2	2
ORL direct, A	OR A to direct byte	2	2
ORL direct, #data	OR immediate to direct byte	3	3
XRL A, Rn	Exclusive-OR Register to A	1	1
XRL A, direct	Exclusive-OR direct byte to A	2	2
XRL A, @Ri	Exclusive-OR indirect RAM to A	1	2
XRL A, #data	Exclusive-OR immediate to A	2	2
XRL direct, A	Exclusive-OR A to direct byte	2	2
XRL direct, #data	Exclusive-OR immediate to direct byte	3	3
CLR A	Clear A	1	1
CPL A	Complement A	1	1
RL A	Rotate A left	1	1
RLC A	Rotate A left through Carry	1	1
RR A	Rotate A right	1	1
RRC A	Rotate A right through Carry	1	1
SWAP A	Swap nibbles of A	1	1
Data Transfer			
MOV A, Rn	Move Register to A	1	1
MOV A, direct	Move direct byte to A	2	2
MOV A, @Ri	Move indirect RAM to A	1	2
MOV A, #data	Move immediate to A	2	2
MOV Rn, A	Move A to Register	1	1
MOV Rn, direct	Move direct byte to Register	2	2
MOV Rn, #data	Move immediate to Register	2	2
MOV direct, A	Move A to direct byte	2	2
MOV direct, Rn	Move Register to direct byte	2	2
MOV direct, direct	Move direct byte to direct byte	3	3
MOV direct, @Ri	Move indirect RAM to direct byte	2	2

Mnemonic	Description	Bytes	Clock Cycles
MOV direct, #data	Move immediate to direct byte	3	3
MOV @Ri, A	Move A to indirect RAM	1	2
MOV @Ri, direct	Move direct byte to indirect RAM	2	2
MOV @Ri, #data	Move immediate to indirect RAM	2	2
MOV DPTR, #data16	Load DPTR with 16-bit constant	3	3
MOVC A, @A+DPTR	Move code byte relative DPTR to A	1	3
MOVC A, @A+PC	Move code byte relative PC to A	1	3
MOVX A, @Ri	Move external data (8-bit address) to A	1	3
MOVX @Ri, A	Move A to external data (8-bit address)	1	3
MOVX A, @DPTR	Move external data (16-bit address) to A	1	3
MOVX @DPTR, A	Move A to external data (16-bit address)	1	3
PUSH direct	Push direct byte onto stack	2	2
POP direct	Pop direct byte from stack	2	2
XCH A, Rn	Exchange Register with A	1	1
XCH A, direct	Exchange direct byte with A	2	2
XCH A, @Ri	Exchange indirect RAM with A	1	2
XCHD A, @Ri	Exchange low nibble of indirect RAM with A	1	2
Boolean Manipulation			
CLR C	Clear Carry	1	1
CLR bit	Clear direct bit	2	2
SETB C	Set Carry	1	1
SETB bit	Set direct bit	2	2
CPL C	Complement Carry	1	1
CPL bit	Complement direct bit	2	2
ANL C, bit	AND direct bit to Carry	2	2
ANL C, /bit	AND complement of direct bit to Carry	2	2
ORL C, bit	OR direct bit to carry	2	2
ORL C, /bit	OR complement of direct bit to Carry	2	2
MOV C, bit	Move direct bit to Carry	2	2
MOV bit, C	Move Carry to direct bit	2	2
JC rel	Jump if Carry is set	2	2/3
JNC rel	Jump if Carry is not set	2	2/3
JB bit, rel	Jump if direct bit is set	3	3/4
JNB bit, rel	Jump if direct bit is not set	3	3/4
JBC bit, rel	Jump if direct bit is set and clear bit	3	3/4
Program Branching			
ACALL addr11	Absolute subroutine call	2	3
LCALL addr16	Long subroutine call	3	4
RET	Return from subroutine	1	5
RETI	Return from interrupt	1	5
AJMP addr11	Absolute jump	2	3
LJMP addr16	Long jump	3	4
SJMP rel	Short jump (relative address)	2	3
JMP @A+DPTR	Jump indirect relative to DPTR	1	3
JZ rel	Jump if A equals zero	2	2/3

Mnemonic	Description	Bytes	Clock Cycles
JNZ rel	Jump if A does not equal zero	2	2/3
CJNE A, direct, rel	Compare direct byte to A and jump if not equal	3	3/4
CJNE A, #data, rel	Compare immediate to A and jump if not equal	3	3/4
CJNE Rn, #data, rel	Compare immediate to Register and jump if not equal	3	3/4
CJNE @Ri, #data, rel	Compare immediate to indirect and jump if not equal	3	4/5
DJNZ Rn, rel	Decrement Register and jump if not zero	2	2/3
DJNZ direct, rel	Decrement direct byte and jump if not zero	3	3/4
NOP	No operation	1	1

Notes on Registers, Operands and Addressing Modes:
Rn - Register R0-R7 of the currently selected register bank. @Ri - Data RAM location addressed indirectly through R0 or R1. rel - 8-bit, signed (two's complement) offset relative to the first byte of the following instruction. Used by SJMP and all conditional jumps. direct - 8-bit internal data location's address. This could be a direct-access Data RAM location (0x00-0x7F) or an SFR (0x80-0xFF). #data - 8-bit constant #data16 - 16-bit constant bit - Direct-accessed bit in Data RAM or SFR addr11 - 11-bit destination address used by ACALL and AJMP. The destination must be within the same 2K-byte page of program memory as the first byte of the following instruction. addr16 - 16-bit destination address used by LCALL and LJMP. The destination may be anywhere within the 64K-byte program memory space. There is one unused opcode (0xA5) that performs the same function as NOP. All mnemonics copyrighted © Intel Corporation 1980.

2.1.4.2 On-chip memory

The memory organization of the CIP-51 System Controller is similar to that of a standard 8051. There are two separate memory spaces: program memory and data memory. Program and data memory share the same address space but are accessed via different instruction types. There are 256 bytes of internal data memory and 64k bytes of internal program memory address space implemented within the CIP-51.

2.1.4.2.1 Program memory

The CIP-51 has a 64k byte program memory space. The MCU implements 64 kB of this program memory space as in-system re-programmed Flash memory, organized in a contiguous block from addresses 0x0000 to 0xFFFF. Note: 512 bytes from 0xFE00 to 0xFFFF (C8051F040/1/2/3/4/5 only) of this memory are reserved for factory use and are not available for user program storage.

Program memory is normally assumed to be read-only. However, the CIP-51 can write to program memory by setting the Program Store Write Enable bit (PSCTL.0) and using the MOVX instruction. This feature provides a mechanism for the CIP-51 to update program code and use the program memory space for nonvolatile data storage. Refer to the C8051F04x data sheet (C8051F04xRev1_4.pdf, page 181) for further details.

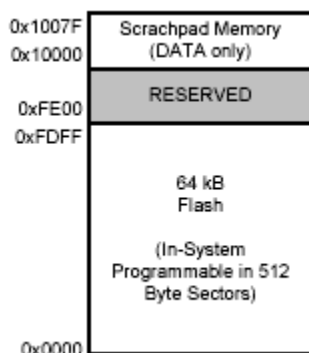


Fig. 2.1.4.2.1 Program Memory (Flash)

2.1.4.2.2 Data memory

The CIP-51 implements 256 bytes of internal RAM mapped into the data memory space from 0x00 through 0xFF. The lower 128 bytes of data memory are used for general purpose registers and scratch pad memory. Either direct or indirect addressing may be used to access the lower 128 bytes of data memory. Locations 0x00 through 0x1F are addressable as four banks of general purpose registers, each bank consisting of eight byte-wide registers. The next 16 bytes, locations 0x20 through 0x2F, may either be addressed as bytes or as 128 bit locations accessible with the direct addressing mode.

The upper 128 bytes of data memory are accessible only by indirect addressing. This region occupies the same address space as the Special Function Registers (SFR) but is physically separate from the SFR space. The addressing mode used by an instruction when accessing locations above 0x7F determines whether the CPU accesses the upper 128 bytes of data memory space or the SFR's. Instructions that use direct addressing will access the SFR space. Instructions using indirect addressing above 0x7F access the upper 128 bytes of data memory.

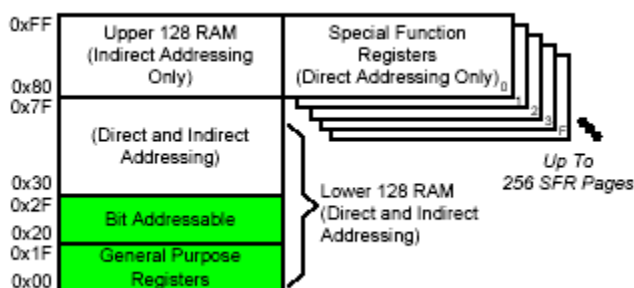


Fig.2.1.4.2.2 Data Memory (RAM)

2.1.4.2.3 External data memory interface

The C8051F04x MCUs include 4k bytes of on-chip RAM mapped into the external data memory space (XRAM), as well as an External Data Memory Interface that can be used to access off-chip memories and memory-mapped devices connected to the GPIO ports.

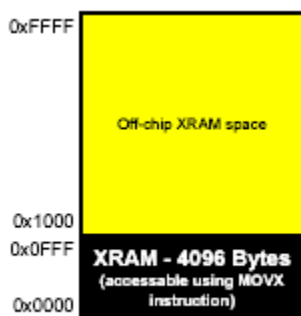


Fig. 2.1.4.2.3 External Data Memory Address Space

2.1.4.2.4 General purpose registers

The lower 32 bytes of data memory, locations 0x00 through 0x1F, may be addressed as four banks of general-purpose registers. Each bank consists of eight byte-wide registers designated R0 through R7. Only one of these banks may be enabled at a time. Two bits in the program status word, RS0 (PSW.3) and RS1 (PSW.4), select the active register bank. This allows fast context switching when entering subroutines and interrupt service routines. Indirect addressing modes use registers R0 and R1 as index registers.

2.1.4.2.5 Bit addressable locations

In addition to direct access to data memory organized as bytes, the sixteen data memory locations at 0x20 through 0x2F are also accessible as 128 individually addressable bits. Each bit has a bit address from 0x00 to 0x7F. Bit 0 of the byte at 0x20 has bit address 0x00 while bit 7 of the byte at 0x20 has bit address 0x07. Bit 7 of the byte at 0x2F has bit address 0x7F. A bit access is distinguished from a full byte access by the type of instruction used (bit source or destination operands as opposed to a byte source or destination).

The MCS-51™ assembly language allows an alternate notation for bit addressing of the form XX.B where XX is the byte address and B is the bit position within the byte. For example, the instruction:

```
MOV    C, 22.3h
```

moves the Boolean value at 0x13 (bit 3 of the byte at location 0x22) into the Carry flag.

2.1.4.2.6 Stack

A programmer's stack can be located anywhere in the 256-byte data memory. The stack area is designated using the Stack Pointer (SP, address 0x81) SFR. The SP will point to the last location used. The next value pushed on the stack is placed at SP+1 and then SP is incremented. A reset initializes the stack pointer to location 0x07; the first value pushed on the stack is placed at location 0x08, which is also the first register (R0) of register bank 1. Thus, if more than one register bank is to be used, the SP should be initialized to a location in the data memory not being used for data storage. The stack depth can extend up to 256 bytes.

2.1.4.2.7 Special function registers (SFRs)

The direct-access data memory locations from 0x80 to 0xFF constitute the special function registers (SFRs). The SFRs provide control and data exchange with the CIP-51's resources and peripherals. The CIP-51 duplicates the SFRs found in a typical 8051 implementation as well as implementing additional SFRs used to configure and access the sub-systems unique to the MCU. This allows the addition of new functionality while retaining compatibility with the MCS-51™ instruction set.

The SFR registers are accessed whenever the direct addressing mode is used to access memory locations from 0x80 to 0xFF. SFRs with addresses ending in 0x0 or 0x8 (e.g. P0, TCON, P1, SCON, IE, etc.) are bit-addressable as well as byte-addressable. All other SFRs are byte-addressable only. Unoccupied addresses in the SFR space are reserved for future use. Accessing these areas will have an indeterminate effect and should be avoided. Refer to the microcontroller's datasheet (page 147) for a detailed description of each register.

SFR Paging

The CIP-51 features SFR paging, allowing the device to map many SFRs into the 0x80 to 0xFF memory address space. The SFR memory space has 256 pages. In this way, each memory location from 0x80 to 0xFF can access up to 256 SFRs. The C8051F04x family of devices utilizes five SFR pages: 0, 1, 2, 3, and F. SFR pages are selected using the Special Function Register Page Selection register, SFRPAGE. The procedure for reading and writing an SFR is as follows:

1. Select the appropriate SFR page number using the SFRPAGE register.
2. Use direct accessing mode to read or write the special function register (MOV instruction).

2.1.5 Hardware setup using an USB debug adapter

The target board is connected to a PC running the Silicon Laboratories IDE via the USB Debug Adapter as shown in Fig. 2.1.5.

1. Connect the USB Debug Adapter to the JTAG connector on the target board with the 10-pin cable.
2. Connect one end of the USB cable to the USB connector on the USB Debug Adapter.
3. Connect the other end of the USB cable to a USB Port on the PC.
4. Connect the ac/dc power adapter to power jack P1 on the target board.

Notes:

- Use the Reset button in the IDE to reset the target when connected using a USB Debug Adapter.
- Remove power from the target board and the USB Debug Adapter before connecting or disconnecting the ribbon cable from the target board. Connecting or disconnecting the cable when the devices have power can damage the device and/or the USB Debug Adapter.

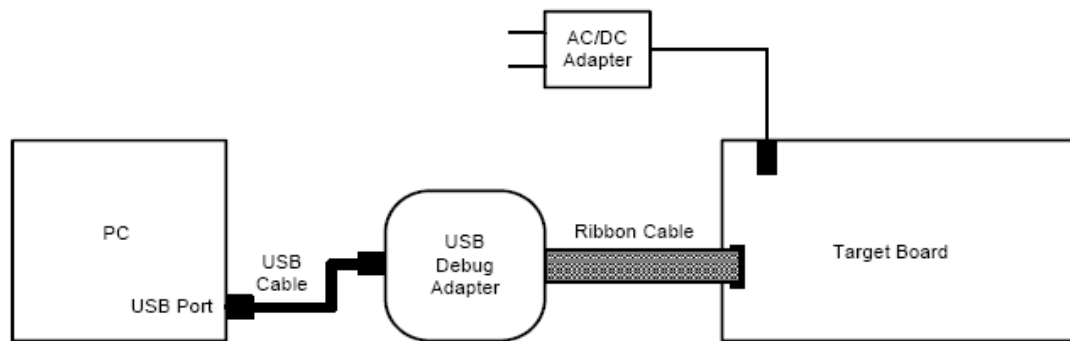


Fig. 2.1.5 Hardware setup using an USB debug adapter

2.2 Development software: Silicon Laboratories IDE

The development environment user interface is composed of three main windows:

- Project Viewer window
 - File View - Used to view and manage files associated with a project.
 - Symbol View- Used to view addresses of symbols used in the project.
- Edit/Debug window
 - Used to compose or edit a selected file within a project.
 - After code has been downloaded, this window is used to view the code execution during a debug session.
- Output window - The output window is composed of tabs used to display information from various processes during development.
 - The build tab displays the output produced by the integrated tools. The user can double click on an error in the build window, and the line of code where the error occurred will be displayed in the editor.
 - The list tab will display the list file generated by the most recent assemble or compile.
 - The tool tab will display the output from a custom tool if the tool output is redirected to the file named "tool.out".

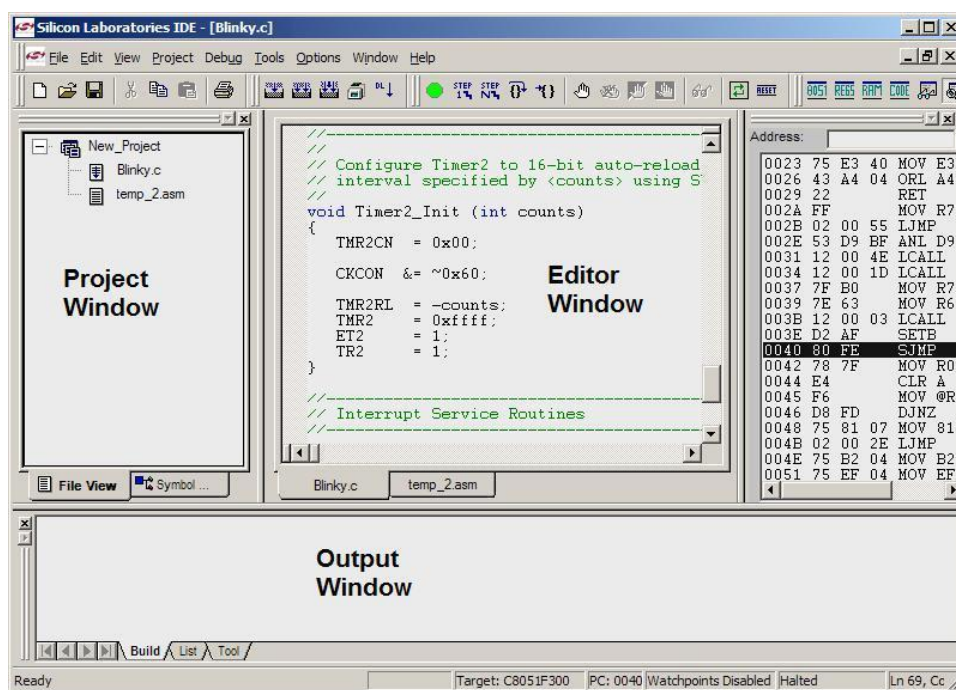


Fig. 2.2 IDE's main windows

In order to execute an application one should follow these steps:

- create a new project: Project -> New Project
- adding one or more source files: Project -> Add Files to Project
- assemble the main source file: Project -> Assemble/Compile File
- build the project: Project -> Build/Make Project
- connect the computer to the target board: Debug -> Connect
- download the object file: Debug -> Download Object File
- execute the application: Debug -> Go

Note:

- one will be able to connect to the target board only after he had finished the hardware setup described in section 2.1.6.

3 Application example

3.1 Requirements

Implement a system that switches a LED on and off with a fixed frequency.

3.2 Proposed solution

3.2.1 Hardware module description

To implement this system we will not need any additional hardware because we can make use of the green LED connected to pin P1.6 as shown in Fig. 3.2.1.

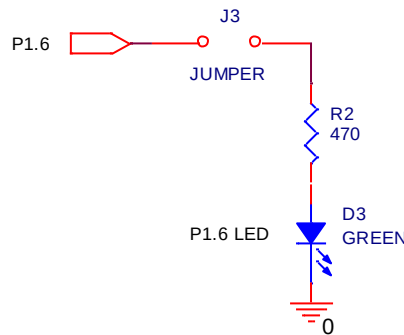


Fig. 3.2.1 Connecting the green LED D3 to the microcontroller

3.2.2 Algorithm description

Generally, all the assembler programs source codes are structured as follows:

- EQUATES
 - this part of the source code will usually contains the include directive (to include the C8051F040.inc file that defines the standard labels for C8051F040 microcontroller) and any other label definitions
- RESET and INTERRUPT VECTORS
 - this part of the source code contains the interrupt vectors definitions (if the current application does not make use of interrupts then it's only necessary to define the reset interrupt vector)
 - refer to the assembler's user's guide (A51.pdf file) for further details
- MAIN PROGRAM CODE SEGMENT
 - this part hosts the main routine of the program
 - the code segment is usually clearly specified
 - refer to the assembler's user's guide (A51.pdf file) for further details regarding the assembly directives (segment, rseg, cseg)
- FUNCTION CODE
 - this part of the source code hosts the sub-routines of the program

The solution proposed in this paper makes use of several sub-routines: `main`, `delay`, `init`.

The main routine (`main`) calls a delay sub-routine and afterwards changes the state of the LED. The last instruction of the main sub-routine jumps back to the `mainLoop` creating an infinite loop.

The delay sub-routine (`delay`) consists of three nested loops (`loop0`, `loop1`, `loop2`). The counters for these three loops are the general purpose registers R5, R6 and R7. Every loop is built-up using the `djnz` instruction (`djnz Rx, label`) which has two purposes: it decrements the value stored in the register and it jumps to the specified label if the new value (the decremented value) is not null. In the example program (`R5 = 0`, `R6 = 0`, `R7 = 02`) the outer loop is executed twice and the two inner loops are executed 256 times.

The initialization sub-routine:

- disables the interrupt mechanism

- This application does not make use of the interrupt mechanism therefore this mechanism is disabled. Moreover, it is recommended to disable this mechanism before disabling the watchdog timer.
- The interrupt mechanism is disabled by resetting the EA flag.
- disables the watchdog timer
 - The watchdog timer is a protection mechanism that resets the microcontroller if, for example, the application freezes. This mechanism is not needed in the current application therefore, it is being disabled.
 - The watchdog timer mechanism is disabled by writing 0xDE and 0xA5 in the WDTCN register.
 - Refer to the microcontroller’s datasheet (C8051F04xRev1_4.pdf file, page 169, page 171) for further details on configuring this mechanism and the definition of the WDTCN register.
- enables the crossbar
 - The crossbar is a digital switch that assigns Port pins to the peripherals in the SOC. The Port pins (only for ports P0, P1, P2 and P3) can be used as general purpose input/output pins or some of the peripherals signals inside the SOC could be mapped at the same Port pins. For example, all the Timers, Counters, Serial Interfaces, ADCs signals can be mapped to the Port pins.
 - This application does not use any peripherals, but the crossbar should be enabled in order to have access to port’s P1 pins. The crossbar is enabled by setting a bit located in the XBR2 register.
 - Refer to the microcontroller’s datasheet (C8051F04xRev1_4.pdf file, page 206, page 216) for further details on configuring the crossbar.
- configures the I/O port P1
 - The application uses pin 6 of port P1 (P1.6) to turn on and off the green LED. Therefore, this pin must be set as output digital pin. The output mode will be Push-Pull. We configure this pin by modifying the value stored in P1MDOUT.
 - Refer to the microcontroller’s datasheet (C8051F04xRev1_4.pdf file, page 219) for the P1MDOUT register definition.

The algorithm is figured in Fig. 3.2.2.

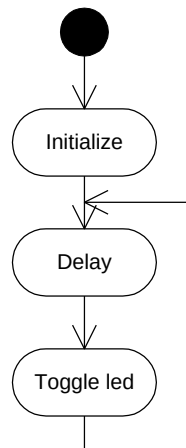


Fig. 3.2.2 Blinky algorithm

4 Exercises

1. Create a new project using the file Z:\Lab8051\Laborator1\blinky.asm and following the steps presented in Section 2.2. Note: The file C8051F040.inc should be stored in the same folder as the source code file.
2. Execute the application.
3. Modify the program in order to decrease the frequency by which the LED changes its state.
4. Compute this frequency considering that the microcontroller runs at 3MHz. Use Table 2.1.4.1.4 to determine the number of cycles for each instruction.
5. Increase the frequency up to the point where the blink of the LED is almost unnoticeable. Note: the critical frequency for the human visual system is approximately 25 – 30Hz.
6. Modify the program so that it increases the frequency by which the LED changes its state over the time.
7. Modify the program so that the LED is lit for a longer period of time. Note: the frequency should be low enough to perceive the blink and the command signal should look like the one in Fig. 4.7.



Fig. 4.7

8. Modify the program so that the LED blinks three times and then turns off for a longer period of time. Note: the command signal should look like the one in Fig. 4.8.



Fig. 4.8

9. Modify the program so that the LED turns on when the S2 switch (on the target board) is pressed and it turns off when the switch is not pressed. The electrical diagram in Fig. 4.9 presents the connection of the switch to the microcontroller.

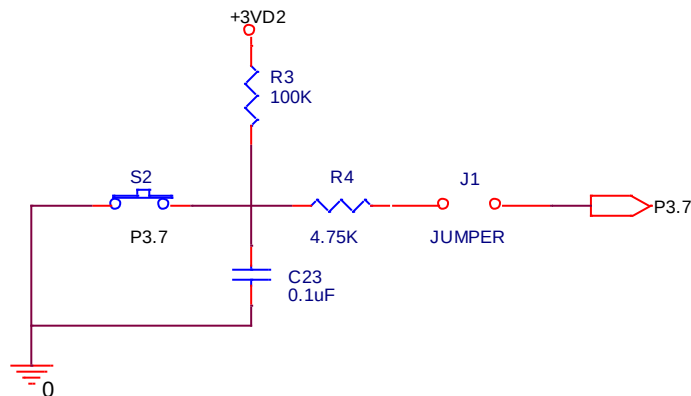


Fig. 4.9 S2 is connected to pin P3.7

5 Appendixes

5.1 Blinky Application source code

```
;-----  
;  
;  
; FILE NAME      : blinky.asm  
; TARGET MCU     : C8051F040  
; DESCRIPTION    : LED blinking.  
;  
; NOTES:  
;  
;-----  
; EQUATES  
;-----  
  
        $include (c8051f040.inc)          ; Include register definition file.  
        GREEN      equ    P1.6            ; Label port P1.6 as GREEN (green led)  
  
;-----  
; RESET and INTERRUPT VECTORS  
;-----  
  
        cseg      AT 0x0000                ; Reset Vector  
        ljmp      main                    ; Locate a jump to the start of code  
                                           ;at the reset vector.  
  
;-----  
; MAIN PROGRAM CODE SEGMENT  
;-----  
  
mainCodeSeg      segment      CODE  
        rseg      mainCodeSeg            ; Switch to this code segment.  
        using     0                      ; Specify register bank for the  
                                           ;following program code.  
  
main:  
    mainLoop:    acall      init  
                  mov       A, #0x02  
                  call      delay  
                  cpl       GREEN  
                  jmp       mainLoop  
  
;-----  
; FUNCTION CODE  
;-----  
  
delay:          mov       R7, A  
    loop1:      mov       R6, #0x00  
    loop0:      mov       R5, #0x00  
                djnz      R5, $  
                djnz      R6, loop0  
                djnz      R7, loop1  
                ret  
  
init:  
                clr       EA                ; Disable global interrupts  
                mov       WDTCN, #0xDE      ; Disable Watch Dog Timer  
                mov       WDTCN, #0xAD  
                mov       SFRPAGE, #CONFIG_PAGE ; Use SFRs in the  
                                           ;configuration Page  
initIOandCross: mov       XBR2, #0x40      ; Enable Crossbar  
                orl       P1MDOUT, #0x40    ; Set P1.6 (GREEN) as digital  
                                           ;output in push-pull mode.  
                clr       GREEN            ; Turn off green led  
                ret  
  
;-----  
; End of file.  
  
END
```